Iniziato	Friday, 21 June 2024, 09:36
Stato	Completato
Terminato	Friday, 21 June 2024, 10:35
Tempo impiegato	59 min. 28 secondi

Domanda **1**

Completo

Punteggio max.: 6,00

Construct a deterministic TM of the kind you prefer, which decides the following language:

$$\mathcal{L} = \{w \in \{0,1\}^* \mid w \text{ contains an even number of 0s and an odd number o}$$

Study the complexity of the TM you have defined.

```
Gamma = {0,1,E,S} is the alphabet, where E is the "empty" char and S is the starting char
Q = {q_init,
       q_0even_1even, /* wrong 1s */
       q_0even_1odd,   /* correct state */
       q_0odd_1even,   /* both 0s and 1s wrong */
       q_0odd_1odd     /* wrong 0s */
       q_halt} is the set of states
delta: QxGamma -> Qx{L,S,R} is the transition function

I'm assuming that the empty string won't be accepted because it does not have an odd number of 1s. Then, after the
q_init, the machine can yet assume the state of 1_even
delta = {
  (q_init, S) -> (q_1even, R)

  /* when 0s are ok and 1s are wrong */
  (q_0even_1even, 1) -> (q_0even_1odd, R) /* both 0s and 1s are ok */
  (q_0even_1even, 0) -> (q_0odd_1even, R) /* both 0s and 1s are wrong */

  /* when 0s are wrong and 1s are ok */
  (q_0odd_1odd, 1)  -> (q_0odd_1even, R) /* both 0s and 1s are wrong */
  (q_0odd_1odd, 0)  -> (q_0even_1odd, R) /* both 0s and 1s are ok */

  /* when 0s and 1s are both wrong */
  (q_0odd_1even, 1) -> (q_0odd_1odd, R)     /* 1s becomes ok, but 0s are still wrong */
  (q_0odd_1even, 0) -> (q_0even_1even, R) /* 0s becomes ok, but 1s are still wrong */

  /* when 0s and 1s are both ok */
  (q_0even_1odd, 1) -> (q_0even_1even, R)
  (q_0even_1odd, 0) -> (q_0odd, 1odd, R)
  (q_0even_1odd, E) -> (q_halt, S) /* end of the execution */
}

It is easy to see that the TM goes through the input from left to right only once, thus its complecity is polynomial (O(n)).
```

Domanda **2**

Completo

Punteggio max.: 7,00

You are required to prove that the following problem $\mathcal{L}$ is in $\mathbf{NP}$. To do that, you can give a TM or define some pseudocode. The language $\mathcal{L}$ includes precisely those binary strings which are encodings of pairs in the form (G, n) where G is an undirected graph having a path which goes through n distinct edges of G, going through each of them exactly once. Can you say anything else about the complexity of this problem?

```
INPUT: (G, n)
CERTIFICATE: P=(p1, ..., pm) which is the path
OUTPUT: True/False
VERIFIER:

/* check if the path is equal to n */
if |P| != n then:
  return False

/* initialization of an array of length n containing only 0s */
A = [0 foreach pi in P]

/* going through the path */
i = 1
j = 2
while j <= n do:
  if (P[i], P[j]) is not in E then:
    return False
  A[i] += 1
  i += 1
  j += 1
end

/* checking if each node in the path was visited only once */
foreach a in A do:
  if a > 1 then:
    return False
return True


Since:
- the input can be represented with a binary string;
- the size of the verifier is polynomially bounded;
- the number of executed instructions is polynomially bounded with he input - O(n);
- each instruction can be exceuted in poly-time (the only non trivial instruction is the initialization of the array that can
executed in poly-time by going through each element of the array);
- intermidiate results are polynomially bounded in length since it is an array of length equal to n;
we can state that this problem is in NP.
```

Domanda **3**

Completo

Punteggio max.: 7,00

The SubsetSum problem is a well-known NP-complete problem defined as follows: given a finite sequence $n_1, \ldots, n_k)$ of natural numbers and a natural number m, determine if there is a subset I of $\{1, \ldots, k\}$ with $\sum_{i \in I} n_i = m$.
Now, consider the following variation to the problem: given a finite sequence $n_1, \ldots, n_k$ of natural numbers and natural numbers m and p, determine if there is a subset I of $\{1, \ldots, k\}$ with $\sum_{i \in I} n_i = m$ and $\sum_{i \notin I} n_i = p$.
To which complexity class does this new problem belong? Prove your claim.

My claim is that the problem is NP-complete.

The problem is for sure in NP and it can be proved by this algorithm that checks in poly-time if a certificate for this problem is correct or not:
INPUT: N=(n1, ..., nk), m, n
CERTIFICATE: I=(i1, ..., ik)
OUTPUT: True/False

◄ LAAI - 21062024 - QUESTIONS - LONG

Vai a...

LAAI - 21062024 - PROBLEMS - LONG ►

```
a = 0
b = 0
foreach ii in  do:
  if ii is in N then:
    a += ii
  else:
    b += ii
end
if a == m and b == p then:
  return True
return False
```

Since SubsetSum is a well-known NP-complete problem and this variation is at least difficult as the original problem, then we can state that it is at least difficult as all the problems in NP (NP-hard). Thus, since it is NP-hard and in NP, we can state that it is NP-complete too.

Commento:
The reduction is given only informally, so imprecisely.